Name: Opeyemi Caleb Akoki

Student ID: 16980763

Module: Software Security 7021SCN

CRITICAL REFLECTION ON 7021SCN SOFTWARE SECURITY PORTFOLIO

SSDF PS.1 - Protect Software from Tampering

Evidence

This repository has two automated pipelines. *.github/workflows/sast.yml* runs Semgrep and Bandit static analysis on every push, and .github/workflows/generate-sbom.yml generates the SBOM. Both use on: push and *runs-on: ubuntu-latest* to execute on a fresh GitHub-hosted runner. See Figure 1.

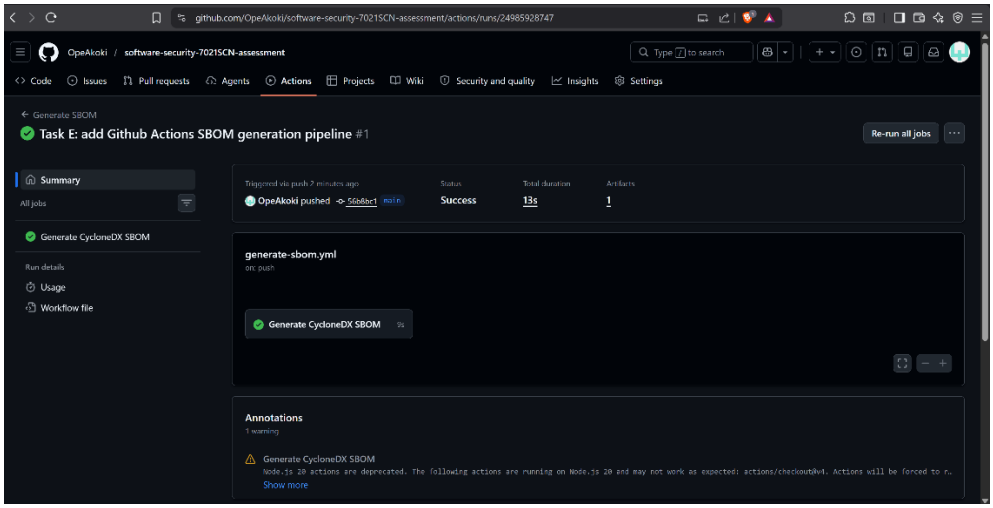


Figure 1: GitHub Actions - Generate SBOM pipeline run (PS.1 evidence)

```
on:
 push:
  branches: ["main"]
```

```
jobs:
 sbom:
  runs-on: ubuntu-latest
```

Argument

The SBOM is not generated on a developer's machine. Every time code is pushed, GitHub spins up a brand new Ubuntu machine, runs Syft to generate the SBOM, and then destroys that machine. No human hand is involved between the commit and the output.

Conclusion

This removes any possibility of tampering with the build process, directly satisfying PS.1.

SSDF PW.4 - Maintain Provenance Data

Evidence

```
570    {
571      "bom-ref": "pkg:pypi/pip@24.0?package-id=5fa4075dce96
572      "type": "library",
573      "author": "The pip developers <distutils-sig@python.or
574      "name": "pip",
575      "version": "24.0",
576      "licenses": [
577        {
```

Figure 2: sbom.cdx.json - pip dependency block showing exact version (PW.4 evidence)

The *sbom.cdx.json* file lists every python package and it's exact version number, as shown above.

Argument

The SBOM acts as an absolute ingredient list for the application. When the next Log4j-style zero-day drops, a team without this file spends days manually checking every server. A team with it runs one query and has an answer in seconds. Every hidden, nested dependency is recorded.

Conclusion

By automatically capturing the exact version of every component during the build phase, strict provenance records are maintained as required by PW.4.

SSDF RV.1 - Identify and Confirm Vulnerabilities on an Ongoing Basis

Evidence

Evidence The pipeline generates the SBOM using format: *cyclonedx-json*. Grype ingested this file and returned two findings with GHSA identifiers and severity ratings. See Figure 3.

```
(venv) userskokey@Opeyemi:/mnt/c/Users/USER/software-security-7021SCN-assessment$ dock
er run --rm -v $(pwd)/task-e-SBOM/sbom:/sbom anchore/grype sbom:/sbom/sbom.cdx.json
NAME   INSTALLED   FIXED IN   TYPE    VULNERABILITY        SEVERITY   EPSS          RISK

pip    24.0        25.3       python  GHSA-4xh5-x5gv-qwph  Medium     < 0.1% (5th)  < 0.1

pip    24.0        26.0       python  GHSA-6vgw-5pg2-w6jp  Low        < 0.1% (7th)  < 0.1
(venv) userskokey@Opeyemi:/mnt/c/Users/USER/software-security-7021SCN-assessment$
```

Figure 3: Grype scan output against *sbom.cdx.json* - two pip vulnerabilities identified (RV.1 evidence)

Argument

A PDF sits on a desk and cannot be queried by anything. A standardised CycloneDX JSON file is different tools like Grype, Aqua Trivy, and enterprise SIEMs can ingest it automatically and cross-reference every component against live CVE databases without human involvement. It is not just a file, it is a feed that security tooling can act on continuously.

Conclusion

Formatting the SBOM as machine-readable data allows vulnerability scans to be automated against updated CVE databases on an ongoing basis, satisfying RV.1.

Threat Modelling and SDLC – Task B

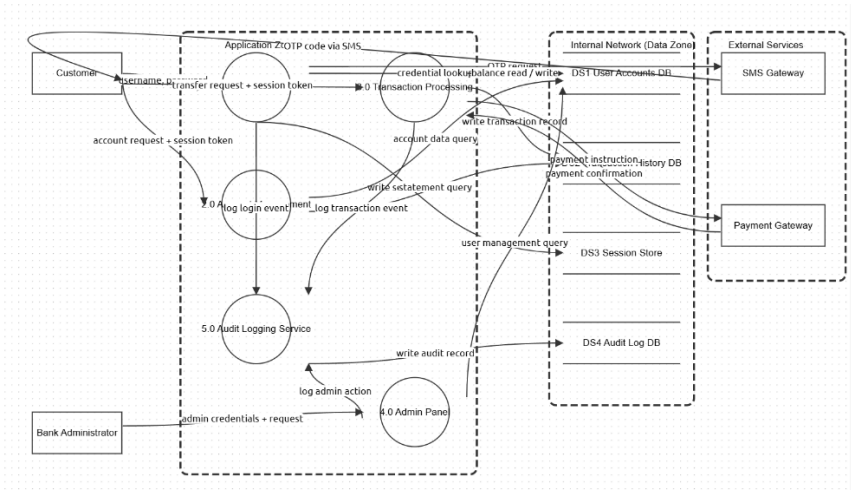


Figure 4. NovaTrust Bank Mobile Application - Threat Dragon DFD with three trust boundary zones (Task B evidence)

Evidence

The STRIDE analysis in *task-b-threatmodel/threat-model.md* identifies eleven threats across all six categories, each linked to a specific data flow in the DFD above.

Argument

STRIDE surfaces design-level flaws that Semgrep and ZAP cannot detect. The cookie forgery vulnerability has no code pattern to scan for. It only appears when you ask the Spoofing question: who can pretend to be someone else? The limitation of STRIDE is that it identifies threat categories but does not rank them by likelihood or business impact, so without additional risk scoring a team can end up treating a theoretical low-probability threat the same as a near-certain critical one.

Conclusion

By applying STRIDE at the design stage, security requirements are defined before any code is written rather than discovered after vulnerabilities reach production.

Dynamic Analysis & Exploit Development – Task D

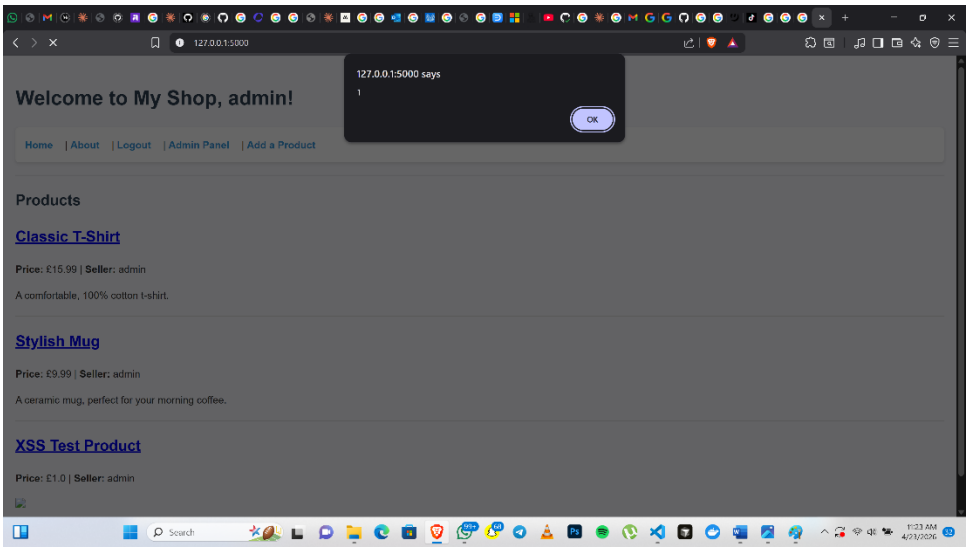


Figure 5: Stored XSS payload firing on the My Shop webapp (Task D evidence)

Evidence

Four vulnerabilities were exploited manually against the local webapp: cookie forgery, SQL injection, stored XSS, and privilege escalation. Screenshots are in *task-d-dynamictesting/evidence/*. OWASP ZAP returned 10 alerts including SQL injection, CSRF, and absent security headers. See Figure 5.

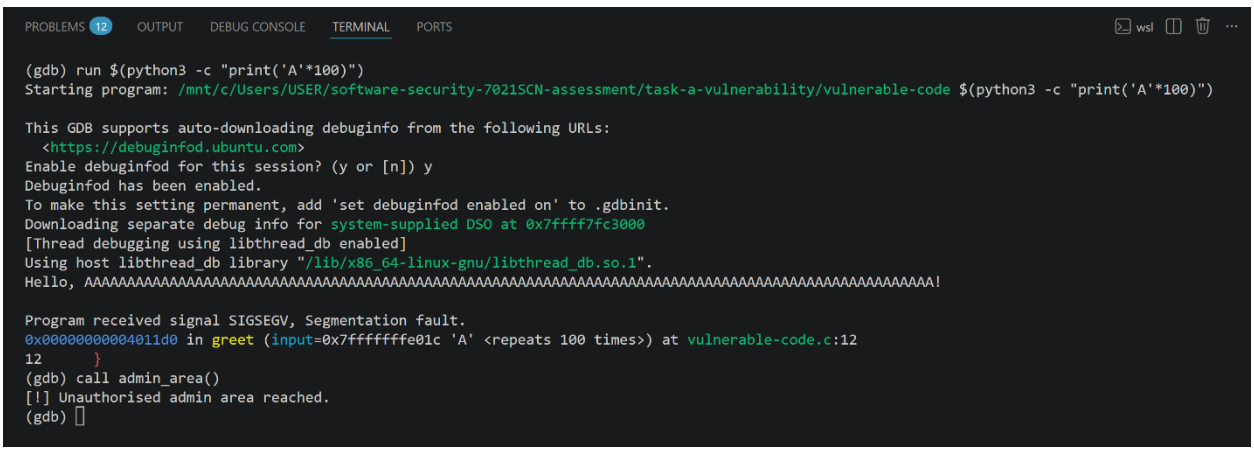
Argument

ZAP confirmed the SQL injection findings but missed cookie forgery and privilege escalation entirely. Both are logic flaws with no detectable syntax pattern. They test what they can measure, not what they cannot anticipate. Manual testing is what closes that gap.

Conclusion

SAST and DAST together do not replace manual penetration testing. The most critical vulnerability in this application was invisible to every automated tool used.

Vulnerability Discovery and Remediation – Task A



```
PROBLEMS 12   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   wsl

(gdb) run $(python3 -c "print('A'*100)")
Starting program: /mnt/c/Users/USER/software-security-7021SCN-assessment/task-a-vulnerability/vulnerable-code $(python3 -c "print('A'*100)")

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0x7ffff7fc3000
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Hello, AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA!

Program received signal SIGSEGV, Segmentation fault.
0x0000000000004011d0 in greet (input=0x7fffffffe01c 'A' <repeats 100 times>) at vulnerable-code.c:12
12      }
(gdb) call admin_area()
[!] Unauthorised admin area reached.
(gdb)
```

Figure 6: GDB redirecting execution to admin_area() via buffer overflow (Task A evidence)

Evidence

task-a-vulnerability/vulnerable-code.c contains a deliberate buffer overflow where strcpy copies user input into a 32-byte buffer with no length check. The fix is in *vulnerable-code-fixed.c*. See Figure 6.

Argument

Passing 100 bytes of input caused an immediate segmentation fault. In GDB, triggering the overflow redirected execution to *admin_area()*, a restricted function the attacker should never reach. The fix replaces *strcpy* with *snprintf* and adds an explicit length check. Compiler flags including *-fstack-protector-strong* and *-z noexecstack* provide runtime defence in depth.

Conclusion

Fixing the root cause in source eliminates the vulnerability entirely. Compiler flags reduce exploitability but an unsafe function call remains unsafe regardless of how the binary is compiled.

Repository link: https://github.com/OpeAkoki/software-security-7021SCN-assessment

AI Usage Table

Tool	How it was used in this assignment
Claude (Anthropic)	Paraphrasing, grammar checking, improving clarity of written sections, and suggesting structural improvements to documentation.

References

Anchore (2024) Grype: A vulnerability scanner for container images and filesystems. Available at: https://github.com/anchore/grype (Accessed: 23 April 2026).

Anchore (2024) Syft: CLI tool and library for generating a software bill of materials from container images and filesystems. Available at: https://github.com/anchore/syft (Accessed: 23 April 2026).

CycloneDX (2024) CycloneDX Specification Overview. Available at: https://cyclonedx.org/specification/overview/ (Accessed: 23 April 2026).

MITRE (2023) CWE-89: Improper Neutralisation of Special Elements used in an SQL Command ('SQL Injection'). Available at: https://cwe.mitre.org/data/definitions/89.html (Accessed: 23 April 2026).

MITRE (2023) CWE-121: Stack-based Buffer Overflow. Available at: https://cwe.mitre.org/data/definitions/121.html (Accessed: 23 April 2026).

National Institute of Standards and Technology (2022) Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities, SP 800-218. Available at:

https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-218.pdf (Accessed: 23 April 2026).

OWASP (2021) *OWASP Top Ten*. Available at: https://owasp.org/www-project-top-ten/ (Accessed: 23 April 2026).

OWASP (2024) *OWASP Threat Dragon*. Available at: https://owasp.org/www-project-threat-dragon/ (Accessed: 23 April 2026).

OWASP (2024) *OWASP ZAP (Zed Attack Proxy)*. Available at: https://www.zaproxy.org/ (Accessed: 23 April 2026).

Pallets (2024) *Flask Documentation*. Available at: https://flask.palletsprojects.com/ (Accessed: 23 April 2026).

Semgrep (2024) *Semgrep Documentation*. Available at: https://semgrep.dev/docs/ (Accessed: 23 April 2026).